

第二次挑战报告

本报告比较共享前缀与分散前缀请求的缓存效果，并结合 SGLang 源码梳理请求处理流程。实验中，两组请求均全部成功；共享前缀命中率为 96.97%，显著减少 Prefill 输入计算量。

任务一 实验环境与设计

项目	配置
硬件	NVIDIA GeForce RTX 4090 24 GB
驱动与工具链	驱动 580.76.05；CUDA Toolkit 13.0
软件版本	Python 3.10.8；PyTorch 2.11.0；SGLang 0.5.14；Ray 2.56.0
模型	Qwen/Qwen3-0.6B
服务配置	上下文 4096；mem_fraction_static=0.5；Radix Cache 开启；overlap 调度开启；stream_interval=1

固定参数。两组各 32 条请求，客户端最大并发为 8；每条输入 2112 个 token，生成 16 个 token。使用原生 POST /generate，提交 input_ids 并开启 stream=true。采样参数为 temperature=0、max_new_tokens=16、ignore_eos=true、sampling_seed=2026。

对照负载。共享组采用 2048-token 公共前缀与 64-token 独立后缀，后缀首 token 互不相同。分散组复制对应请求，仅把第一个 token 改为各请求互不相同的 token，以阻止前缀复用；其余输入、请求编号和提交顺序相同。该合成负载用于测量缓存效果，不评价回答质量。

执行顺序。先用短请求预热服务，再测分散组，最后测共享组。每组开始前调用 /flush_cache 并核对成功响应；共享组清缓存后，先发送仅含公共前缀的预热请求，等待完成再测量。预热不计入结果。客户端以信号量限制并发，组计时结束后才写日志和结果文件。

指标与成功判定

成功要求 HTTP 200、收到 SSE [DONE]、输入长度为 2112、输出长度为 16，且流式累计 token 计数符合检查。成功率为成功请求数除以 32。

缓存命中率 = $\Sigma \text{cached_tokens} / \Sigma \text{prompt_tokens}$ ；按题目口径统计的 Prefill token 数 = $\Sigma(\text{prompt_tokens} - \text{cached_tokens})$ 。请求吞吐量与输出吞吐量分别为成功请求数、输出 token 总数除以组测量墙钟时间。

TTFT 为请求提交至收到首个输出 token；TPOT 为首末输出 token 接收时间差除以 15；端到端延迟为提交至收到 [DONE]。p50、p95 使用线性插值分位数。单请求延迟从取得信号量后计时，不包含客户端并发等待；组墙钟时间包含该等待。延迟包含服务排队与网络传输，并非纯 GPU 计算时间。

任务一 实验结果与分析

测量时间为北京时间 2026 年 10 月 3 日 10:47:33。本次为单次对照运行；下表保留两位小数。

指标	分散前缀	共享前缀
成功请求数与成功率	32/32 100%	32/32 100%
输入 token 总数	67,584	67,584
缓存 token 总数	0	65,536
缓存命中率	0%	96.97%
Prefill token 总数	67,584	2,048
请求吞吐量 requests/s	29.95	105.49
输出吞吐量 tokens/s	479.12	1687.89
TTFT p50 / p95 ms	141.20 / 198.40	32.06 / 39.85
TPOT p50 / p95 ms	8.48 / 13.63	2.60 / 3.44
端到端延迟 p50 / p95 ms	262.03 / 277.75	72.58 / 80.81

前缀复用的计算量变化

共享组每条命中 2048 个 token，只需计算后缀的 64 个 token，因此总缓存量为 $32 \times 2048 = 65,536$ ，总 Prefill 量为 $32 \times 64 = 2,048$ ，命中率为 $2048 / 2112 = 96.97\%$ 。分散组没有前缀命中，总 Prefill 量为 $32 \times 2112 = 67,584$ 。共享组减少了 96.97% 的 Prefill token，输出吞吐量约为分散组的 3.52 倍，TTFT p50 下降约 77.29%。

因果注意力使每个位置的 KV 与此前 token 有关。相同前缀可复用已计算 KV；如果首个 token 不同，后面即使出现相同的 token，也不能把其 KV 当作相同上下文下的缓存直接复用。共享组计算未缓存后缀时仍需访问前缀 KV，复用减少的是前缀重复计算，不是丢弃前缀上下文。

共享组 TPOT 也有所降低，但前缀缓存并未消除逐步 Decode。减少 Prefill 后的资源竞争与调度变化可能影响 TPOT，这属于机制推断；仅凭一次运行不能确定所有原因或保证其他负载下仍有相同比例的提升。

复现与结果依据

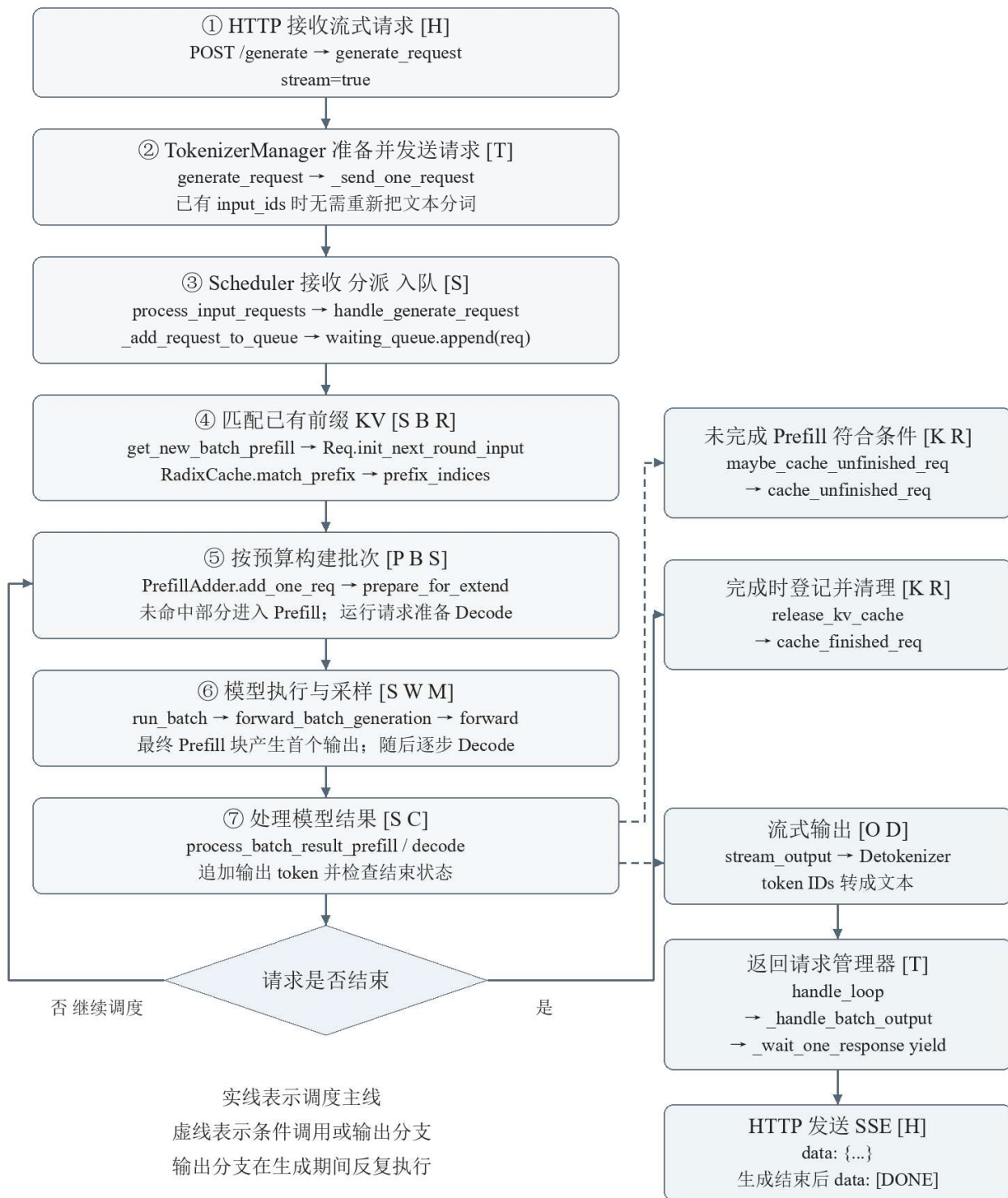
服务就绪后，在已激活的云端虚拟环境中运行：

```
python src/target1/prefix_benchmark.py --base http://127.0.0.1:30000 --model /root/autodl-tmp/models/Qwen3-0.6B --results replay-results
```

实验脚本位于 src/target1/prefix_benchmark.py，提交结果位于 results/target1/，原始运行编号为 run-20261003T024733.130495Z。config.json 与 server_info.json 记录环境和配置，comparison.csv 与两组 summary.json 支持对照表，requests.csv 与 request-01.json 至 request-32.json 支持逐请求核对。validation.json 中 assignment_checks_passed=true。复跑时使用 replay-results/，脚本会自动创建新的 run 子目录，避免覆盖已提交数据。

任务二 SGLang 请求处理流程

SGLang 0.5.14 普通非分离式生成请求。按题目指定的 `event_loop_normal` 梳理逻辑主线：本次实验实际启用了 `overlap` 调度，图中合并了批次准备与辅助调用。



请求流程说明

请求进入等待队列。HTTP 入口调用 `TokenizerManager.generate_request`，规范化参数并准备内部请求；本次实验直接提交 `input_ids`，因此不需要重新对输入文本分词。`_send_one_request` 将请求发送给 `Scheduler`。普通循环接收并分派请求，`handle_generate_request` 创建 `Req`，必要检查完成后，`_add_request_to_queue` 将其加入 `waiting_queue`，等待调度。

前缀匹配减少 Prefill。构建 Prefill 批次时，`Req.init_next_round_input` 调用 `RadixCache.match_prefix`，将已缓存 KV 的存储索引保存为 `prefix_indices`。未缓存输入长度等于完整输入长度减去命中前缀长度；`PrefillAdder` 按资源预算接纳请求，`prepare_for_extend` 组织未缓存输入，并保留访问已有 KV 所需的信息。本次共享组每条输入 2112 个 token，命中 2048 个，只需对剩余 64 个做 Prefill。因果注意力使后续 KV 依赖此前 token，所以首个 token 不同的分散组无法复用共享前缀。

模型生成与缓存登记。`run_batch` 调用 worker 的 `forward_batch_generation`，由模型执行器完成 forward 和采样。最后一个 Prefill 块通常产生首个输出 token，随后 Decode 逐步生成；未完成的 Prefill 分块则继续调度。结果处理器追加输出并检查结束条件。符合条件的未完成请求调用 `cache_unfinished_req`；完成请求经 `release_kv_cache` 调用 `cache_finished_req`。`RadixCache` 登记 token 序列与已计算 KV 的索引，处理重复分配及引用，再释放请求资源。缓存只覆盖已计算且可提交的 KV；刚采样出的 token 不一定已有 KV，也不是每个 Decode 步都调用未完成缓存插入。

流式返回。结果处理后的 `stream_output` 将可输出内容交给 `Detokenizer`，把 token IDs 转成文本后返回 `TokenizerManager`。`handle_loop` 接收结果，`_handle_batch_output` 更新请求状态并唤醒等待者，`_wait_one_response` 逐步 yield。HTTP 将结果编码为 SSE 的 `data: {...}`，生成结束后发送 `data: [DONE]`。输出在生成期间反复进行，并非逐字返回，也不必等请求完成后才开始。

图中源码定位

以下路径均以 `sglang/srt/` 为起点；行号对应本次云端源码副本。

[H] `entrypoints/http_server.py:769`；[T] `managers/tokenizer_manager.py:576、1319、1425、1824、1839`。

[S] `managers/scheduler.py:1505、1998、2258、2702、3145、3367`；`waiting_queue.append(req)` 位于 2265。

[B] `managers/schedule_batch.py:1123、2008`；[P] `managers/schedule_policy.py:85、858`。

`match_prefix_for_req` 是另一前缀匹配入口，与 `Req.init_next_round_input` 不构成必然的先后调用。

[R] `mem_cache/radix_cache.py:358、438、485`；[K] `mem_cache/common.py:113、629`。

[W] `managers/tp_worker.py:482`；[M] `model_executor/model_runner.py:2896`。

[C] `managers/scheduler_components/batch_result_processor.py:178、636`；[O]

`managers/scheduler_components/output_streamer.py:91`；[D] `managers/detokenizer_manager.py`。

作业感受

(1) 简述下你是如何完成第二次挑战的。

完成第二次挑战时，我先恢复第一次挑战使用的云端环境，检查 GPU、模型文件和软件版本，补装 Ray 并启动 SGLang 服务，确认模型能够正常接收请求。

任务一中，我在 AI 辅助下编写并运行实验脚本，设置共享前缀和分散前缀两组请求，统一输入长度、输出长度及并发数。通过清缓存和共享前缀预热控制实验条件，再根据流式返回记录统计缓存命中率、Prefill token 数、吞吐量和延迟。两组请求均全部成功，共享组缓存命中率为 96.97%，Prefill token 总数由分散组的 67,584 降至 2,048。

任务二中，我提取并下载关键源码，在 AI 的讲解和阅读建议下，梳理请求接收、等待队列、前缀匹配、Prefill 与 Decode、缓存登记和流式返回的关系。遇到不理解的概念时，我通过追问和用自己的话复述来检查理解，最后将实验结果、流程图和说明整理成报告，并记录 AI 提供的帮助。

(2) 你觉得哪部分工作最困难，你是如何克服这个困难的。(禁止 AI 辅助撰写)

我觉得阅读源码部分最困难，因为源码的代码规模量很大，所以阅读量很大，再加上我对于 python 的学习还不够深入，所以阅读起来还是有困难。我是通过先让 AI 帮我找到用于回答问题关键代码，定位之后我再去对应的地方寻找并阅读，这样可以缩小阅读范围，能大致明白代码的意思，遇到看不懂的，我也会求助 AI，让它帮我解释一下代码的含义，帮我理解

(3) 你觉得类似的从 0 到 1 的科研工作，需要包含哪些方面的研究，对你今后的科研工作有什么启发(禁止 AI 辅助撰写)

我觉得学习理解一个新的理论、模型不能只关注最后的结果，也应该弄清楚它是如何一步步实现的，例如这次的任务，大模型收到输入后，请求如何进入队列，怎样匹配并复用已有的 KV，如何完成 Prefill 和 Decode，以及怎样保存可复用的 KV 并逐步返回生成结果。对于以后的科研工作，我也会注重这种过程性分析的能力培养，遇到问题时先梳理各个步骤及其联系，再逐步定位和理解关键环节。